

MINI PROJECT REPORT
on
SCRAPING DATA FROM GOOGLE MAPS USING PYTHON

Submitted in partial fulfilment of the Academic requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

CSE – CYBERSECURITY

Submitted by

D.SAI NIKHITHA

[23D41A6213]

Under the esteemed guidance of

Dr. Tamilarasan

Professor



DEPARTMENT OF CSE – CYBERSECURITY

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC, accredited by NBA&NAAC, Affiliated to JNTUH)

Sheriguda, Ibrahimpatnam, Ranga Reddy-501510

2025-2026

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY

(An Autonomous Institute)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTU Hyderabad)

DEPARTMENT OF CSE – CYBERSECURITY



CERTIFICATE

This is to certify that the Mini Project entitled “**SCRAPING DATA FROM GOOGLE MAPS USING PYTHON**” is submitted in partial fulfilment for the award of Degree of Bachelors of Technology in CSE - Cybersecurity of Sri Indu College of Engineering and Technology, Hyderabad for the academic year 2025-2026. This is a record of the project work done by **D.Sai Nikhitha (23D41A6213)** which has been approved as it satisfies academic requirements in respect of the work prescribed for III YEAR, II-SEMESTER of B. TECH course.

Dr. TAMILARASAN
(INTERNAL GUIDE)

Dr. T. SRAVANTI
(HoD)

EXTERNAL EXAMINER

DECLARATION

We, hereby declare that the project work entitled " **SCRAPING DATA FROM GOOGLE MAPS USING PYTHON** " was carried out by us towards the fulfilment of the curriculum required for the award of B.Tech III Year II Semester at SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY. This project is an original work done by us and not submitted from another university or institution for the award of any other degree.

Signature of candidate:

i.

ACKNOWLEDGEMENT

With great pleasure we want to take this opportunity to express our heartfelt gratitude to all the people who helped in making this project a success. We thank the almighty for giving us courage & perseverance in completing the project.

We are thankful to Principal **Dr. G. SURESH**, for giving us the permission to carry out this project and for providing necessary infrastructure and labs.

We are highly indebted to **Dr. T. SRAVANTI**, Head of the Department of CSE-Cyber Security for providing valuable guidance at every stage of this project.

We are grateful to our internal project guide, **Dr. TAMILARASAN**, Professor, for his constant motivation and guidance given by him during the execution of project work.

We would like to thank our project coordinator **Mr. P. PRASHANTH**, Asst. Professor for his continuous guidance and support

We would like to thank the Teaching & Non-Teaching staff of the Department of CSE-Cyber Security for sharing their knowledge with us.

Last but not the least we express our sincere thanks to everyone who helped directly or indirectly for the completion of the project.

D.SAI NIKHITHA

[23D41A6213]

ABSTRACT

In the modern digital age, location-based data plays a crucial role in business intelligence, urban planning, logistics, and many other domains. Google Maps is one of the most widely used platforms for accessing geospatial and location-based information. However, extracting this data manually is time-consuming and inefficient. This project focuses on automating the process of extracting business details, addresses, contact information, and geographical coordinates from Google Maps using Python. By utilizing tools such as Selenium and BeautifulSoup, along with browser automation, the system can extract structured data efficiently and store it in usable formats. This automation helps in building datasets for market analysis, lead generation, and data analytics purposes.

CONTENTS

TOPICS	PAGE.NO
LIST OF FIGURES	I
1. INTRODUCTION	01
1.1 INTRODUCTION	01
1.2 PROBLEM STATEMENT	02
1.3 SIGNIFICANCE AND RELAVANCE OF WORK	02
1.4 OBJECTIVES	02
1.5 METHODOLOGIES	02
2. LITERATURE SURVEY	03
2.1 WEB SCRAPING FOR DATA COLLECTION	03
2.2 SELENIUM FOR BROWSER AUTOMATION	03
2.3 BEAUTIFULSOUP FOR HTML PARSING	03
2.4 GOOGLE MAPS AS A DATA SOURCE	03
2.5 AUTOMATED DATA EXTRACTION AND ANALYSIS	03
3. EXISTING SYSTEM	04
3.1 INTRODUCTION	04
3.2 LIMITATIONS	04
4. PROPOSED SYSTEM	05
4.1 INTRODUCTION	05
4.2 ADVANTAGES	06
5. REQUIREMENTS AND SPECIFICATIONS	07
5.1 INTRODUCTION	07
5.2 HARDWARE REQUIREMENTS	07
5.3 SOFTWARE REQUIREMENTS	07

6. DESIGN ANALYSIS	08
6.1 INTRODUCTION	08
6.2 REQUIREMENTS SPECIFICATION	08
6.2.1 FUNCTIONAL REQUIREMENTS	08
6.2.2 NON-FUNCTIONAL REQUIREMENTS	09
6.2.3 PERFORMANCE REQUIREMENT	10
6.3 INTRODUCTION TO PYTHON	10
6.3.1 INTRODUCTION TO SELENIUM	10
6.3.2 INTRODUCTION TO BEAUTIFULSOUP	11
6.4 SYSTEM DESIGN	11
6.4.1 SYSTEM ARCHITECTURE	12
6.5 DATA FLOW DIAGRAMS	12
6.5.1 DATA FLOW DIAGRAM LEVEL 0	13
6.5.2 DATA FLOW DIAGRAM LEVEL 1	13
6.5.3 DATA FLOW DIAGRAM LEVEL 2	14
6.6 UML DIAGRAMS	15
6.6.1 CLASS DIAGRAM	15
6.6.2 USE CASE DIAGRAM	15
6.6.3 SEQUENCE DIAGRAM	16
6.6.4 COLLABORATIVE DIAGRAM	16
7. IMPLEMENTATION AND RESULT	17
7.1 INTRODUCTION	17
7.2 CODE FOR IMPLEMENTATION	18
7.3 OUTPUT	19
8. CONCLUSION AND FUTURE ENHANCEMENT	22
8.1 CONCLUSION	22
8.2 FUTURE ENHANCEMENT	22
REFERENCES	23

LIST OF FIGURES

FIG NO.	PAGE.NO.
FIG 6.1. PROJECT FLOW	10
FIG 6.2. SYSTEM ARCHITECTURE	11
FIG 6.3. DFD LEVEL 0	12
FIG 6.4. DFD LEVEL 1	13
FIG 6.5. DFD LEVEL 2	13
FIG 6.6. CLASS DIAGRAM	14
FIG 6.7. USE CASE DIAGRAM	14
FIG 6.8. SEQUENCE DIAGRAM	15
FIG 6.9. COLLABRATIVE DIAGRAM	15
FIG 7.1. PYTHON START	17
FIG 7.2. HOME PAGE	17
FIG 7.3. USER REGISTRATION	17
FIG 7.4. REGISTRATION SUCCESS	18
FIG 7.5. USER LOGIN	18
FIG 7.6. OTP VERIFICATION MAIL	18
FIG 7.7. OTP AUTHENTICATION	19

CHAPTER-01

INTRODUCTION

1.1 INTRODUCTION

Google Maps has become one of the most popular platforms for accessing location-based information, providing details about businesses, services, landmarks, and geographical locations worldwide. In today's data-driven environment, organizations rely heavily on accurate location data for market research, customer analysis, logistics planning, and business development. However, collecting large amounts of data manually from Google Maps is a time-consuming and inefficient process. With the rapid growth of businesses and online services, there is an increasing need for automated techniques that can gather and organize location-based information quickly and accurately.

Web scraping is a powerful technique used to extract data from websites automatically. This project focuses on scraping data from Google Maps using Python, a widely used programming language known for its simplicity and extensive library support. By utilizing tools such as Selenium and BeautifulSoup, the system can interact with web pages, navigate through search results, and extract valuable information such as business names, addresses, phone numbers, ratings, review counts, and geographical coordinates. The collected data can then be stored in structured formats like CSV files or databases for further analysis and processing.

The proposed system aims to provide an efficient and scalable solution for extracting location-based data without relying entirely on paid APIs or manual methods. Automation reduces human effort, improves accuracy, and enables the collection of large datasets within a short period. Such data can be useful for various applications, including lead generation, market analysis, competitor research, urban planning, and business intelligence. By implementing a Python-based web scraping system, users can efficiently gather and manage location-related information while saving time and resources. This project demonstrates the practical use of web automation technologies to simplify data collection and support informed decision-making in various domains.

1.2 PROBLEM STATEMENT

- a. Manual collection of business information from Google Maps is time-consuming, error-prone, and inefficient for large-scale data requirements. Additionally, the official Google Maps APIs have limitations such as request quotas, restricted access to certain data fields, and usage costs.
- b. To overcome these issues, the proposed system uses Python-based web scraping techniques with Selenium and BeautifulSoup to automatically extract business details, addresses, phone numbers, ratings, reviews, and geographical coordinates from Google Maps in a structured and efficient manner.

1.3 SIGNIFICANCE AND RELEVANCE OF WORK

- a. The project automates the extraction of location-based information from Google Maps, significantly reducing the time and effort required for manual data collection.
- b. It enables businesses and researchers to gather large volumes of data efficiently for market analysis, competitor analysis, lead generation, and business intelligence purposes.
- c. The system provides structured and organized datasets by extracting business names, addresses, contact details, ratings, and reviews, making data analysis easier.

1.4 OBJECTIVES

- a. To develop an automated web scraping system using Python, Selenium, and BeautifulSoup for extracting business-related information from Google Maps.
- b. To collect and store business names, addresses, phone numbers, ratings, review counts, and geographical coordinates in a structured format such as CSV files or databases.

1.5 METHODOLOGY

- a. The system uses Selenium to automate browser interactions with Google Maps, perform search operations, navigate through listings, and handle dynamic content loading.
- b. BeautifulSoup or lxml is used to parse extracted HTML content and retrieve relevant business information such as names, addresses, phone numbers, ratings, reviews, and coordinates.

CHAPTER-02

LITERATURE SURVEY

2.1 WEB SCRAPING FOR DATA COLLECTION

Web scraping is a technique used to automatically extract information from websites. It eliminates the need for manual data collection and enables users to gather large amounts of data efficiently. Web scraping has become widely used in market research, business intelligence, lead generation, and data analytics applications [1].

2.2 SELENIUM FOR BROWSER AUTOMATION

Selenium is an open-source browser automation framework that allows applications to interact with websites just like human users. It can perform actions such as opening web pages, entering search queries, clicking buttons, and scrolling through dynamic content. Selenium is particularly useful for scraping websites that load content dynamically through JavaScript, making it a suitable choice for Google Maps data extraction [2].

2.3 BEAUTIFULSOUP FOR HTML PARSING

BeautifulSoup is a Python library used for parsing HTML and XML documents. It helps developers extract specific information from web pages by navigating through webpage structures and locating required elements. BeautifulSoup simplifies the process of retrieving business details, contact information, ratings, and reviews from HTML content, making data extraction more efficient and reliable [3].

2.4 GOOGLE MAPS AS A DATA SOURCE

Google Maps is one of the most widely used platforms for accessing location-based information. It contains valuable business data such as names, addresses, phone numbers, ratings, reviews, and geographical coordinates. Extracting this information helps organizations perform market analysis, competitor research, lead generation, and location-based decision-making [4].

2.5 AUTOMATED DATA EXTRACTION AND ANALYSIS

Automated data extraction systems improve efficiency by collecting and organizing information without manual intervention. The extracted data can be stored in structured formats such as CSV files or databases for further analysis. These systems support business intelligence, customer analysis, urban planning, and various research activities by providing accurate and up-to-date location-based datasets [5].

CHAPTER-03

EXISTING SYSTEM

3.1 INTRODUCTION

In the existing system, users generally obtain location-based information from Google Maps either manually or through the official Google Places API. Manual data collection involves searching for businesses on Google Maps and copying details such as business names, addresses, phone numbers, ratings, and reviews. This method is suitable only for small-scale requirements and becomes highly inefficient when a large amount of data needs to be collected.

Another commonly used approach is the Google Places API, which provides structured access to location data. Although the API offers reliable and accurate information, it comes with several limitations. Users must create an API key, configure billing accounts, and comply with request quotas and usage restrictions. Access to advanced features or larger datasets often requires paid plans, which can increase operational costs.

Some users also rely on browser extensions or simple automation scripts to collect Google Maps data. However, these solutions often lack flexibility, scalability, and proper error-handling capabilities. They may fail when website layouts change or when dealing with dynamic content loading. As a result, existing methods are either time-consuming, costly, or incapable of handling large-scale data extraction efficiently, creating the need for a more automated and scalable solution.

3.2 LIMITATIONS

- Manual data collection requires significant time and effort, making it unsuitable for large-scale data gathering.
- Google Places API imposes request quotas and access restrictions, limiting the amount of data that can be extracted.
- API-based solutions may involve additional costs and billing requirements, increasing overall operational expenses.
- Existing tools and methods struggle to handle large volumes of data efficiently, resulting in scalability issues.

CHAPTER-04

PROPOSED SYSTEM

4.1 INTRODUCTION

The proposed system introduces an automated solution for extracting location-based information from Google Maps using Python. It utilizes Selenium for browser automation and BeautifulSoup or xml for parsing web page content. The system automatically searches for specific businesses or locations on Google Maps and retrieves valuable information such as business names, categories, addresses, phone numbers, ratings, review counts, and geographical coordinates. This eliminates the need for manual data collection and improves the efficiency of the entire process.

The scraper is designed to interact with dynamic web content by handling scrolling, page navigation, and data loading operations. It simulates human browsing behavior to ensure smooth data extraction and reduce the chances of detection. The system can process multiple search queries and collect large amounts of data from different geographic locations. Extracted information is cleaned and organized into a structured format, making it easier to use for further analysis and reporting.

The proposed system provides a scalable, cost-effective, and automated approach for collecting Google Maps data without relying heavily on API limitations. It significantly reduces human effort while improving data accuracy and collection speed. The extracted datasets can be used for various applications such as market research, lead generation, business intelligence, competitor analysis, and location-based decision-making. By leveraging Python-based web scraping technologies, the system offers an efficient solution for gathering and managing large volumes of geospatial and business-related information.

Google Maps Data Scraping System offers a reliable and intelligent solution for collecting business information from Google Maps automatically. By integrating Python with Selenium and BeautifulSoup, the system reduces manual intervention and enables fast, accurate, and efficient data extraction. It can collect information from multiple business categories and locations within a short period, making it highly suitable for research and commercial applications.

Unlike paid APIs with request limitations, the proposed system provides an affordable alternative for collecting publicly available business information. The extracted data can be used for market analysis, competitor research, customer targeting, business expansion planning, and lead generation. Overall, the system enhances productivity, improves data accuracy, saves time, and provides valuable insights for informed business decision-making.

The system is highly scalable and can be customized according to user requirements. It supports extracting data from different geographical regions and can handle multiple search queries without affecting performance. The structured output generated in CSV or database format simplifies data management and enables further processing using data analysis and visualization tools.

It efficiently handles dynamic web pages through automated scrolling and navigation, ensuring smooth data extraction. The collected datasets can be used for market research, lead generation, competitor analysis, business intelligence, and location-based decision-making. Overall, the system enhances productivity, increases data reliability, and provides a flexible platform that can be extended for future data analytics and machine learning applications.

4.2 ADVANTAGES

- Automates data extraction, reducing manual effort and saving time.
- Collects large volumes of data efficiently from multiple locations and categories.
- Eliminates API quota limitations, allowing flexible data collection.
- Provides accurate and structured datasets for analysis and reporting.
- Handles dynamic web content effectively using Selenium automation.
- Supports market research, lead generation, and business intelligence applications.
- Improves productivity by collecting large amounts of data quickly.
- Can be extended to integrate with machine learning and business analytics applications.
- Assists organizations in market research, competitor analysis, and strategic decision-making.
- Offers a cost-effective alternative to paid APIs with usage limitations.
- Generates structured datasets that are easy to analyze and manage.
- Supports scraping data from multiple locations and business categories.

CHAPTER-05

REQUIREMENTS AND SPECIFICATIONS

5.1 INTRODUCTION

This section defines the hardware and software requirements needed for the successful development and execution of the system. It outlines the functional and non-functional requirements, along with performance expectations, to ensure the system operates efficiently and meets user needs.

5.2 HARDWARE REQUIREMENTS

Hardware requirements define the essential physical components such as CPU, RAM, storage devices, and GPU needed for efficient software operation, helping ensure proper system configuration and performance.

- | | |
|--------------|-----------------------|
| 1. System | : Pentium IV 2.4 GHz. |
| 2. Hard Disk | : 40 GB. |
| 3. Ram | : 512 MB. |

5.3 SOFTWARE REQUIREMENTS

Software requirements define the essential elements such as programming languages, frameworks, libraries, and databases needed for development, implementation, and smooth functioning of a system, ensuring proper performance and successful deployment.

- | | |
|-------------------------|-----------------|
| 1. Operating system | : Windows |
| 2. Coding Language | : Python |
| 3. Version | : Python 3.7.4 |
| 4. Web Automation Tool | : Selenium |
| 5. HTML Parsing Library | : BeautifulSoup |

CHAPTER-06

DESIGN ANALYSIS

6.1 INTRODUCTION

The design of the Google Maps Data Scraping System focuses on automating the extraction of location-based and business-related information from Google Maps. The system is developed using Python and integrates Selenium for browser automation and BeautifulSoup for data extraction. Users provide search queries such as business categories or locations, and the system automatically navigates through Google Maps to collect relevant information. This approach eliminates the need for manual data collection and improves efficiency in gathering large amounts of data.

The system architecture consists of several modules, including user input, browser automation, data extraction, data processing, and data storage. Selenium handles dynamic web page interactions such as searching, scrolling, and loading additional results. BeautifulSoup or lxml parses the webpage content and extracts business details such as names, addresses, phone numbers, ratings, review counts, and geographical coordinates. The extracted information is then validated and cleaned to ensure consistency and accuracy.

The final stage of the design involves organizing and storing the collected data in structured formats such as CSV files or databases. The modular architecture improves scalability, maintainability, and performance, allowing the system to handle multiple search queries and large datasets efficiently. By automating the entire workflow, the system reduces human effort, minimizes errors, and provides reliable datasets for market research, lead generation, business intelligence, and analytical applications. The design ensures efficient data collection while maintaining flexibility for future enhancements and additional functionalities.

6.2 REQUIREMENTS SPECIFICATION

6.2.1 FUNCTIONAL REQUIREMENTS

Functional Requirements define the functions of a software system and describe how the system must behave when presented with specific inputs or conditions. These requirements include data extraction, processing, storage, automation, and reporting functionalities necessary for the successful operation of the Google Maps Data Scraping System.

1. The system must allow users to enter business categories, keywords, or location names as search inputs.
2. The system must automatically access Google Maps and perform searches .

3. The system must extract business information such as names, addresses, phone numbers, ratings, review counts, and geographical coordinates.
4. The system must handle dynamic webpage elements, including scrolling, pagination, and content loading.
5. The system must clean, validate, and organize the extracted data to improve accuracy and consistency.
6. The system must store the collected information in structured formats such as CSV files or databases.
7. The system must support data extraction from multiple locations and business categories.
8. The system must generate organized datasets that can be used for market research, lead generation, business intelligence, and analytical purposes.

6.2.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements are the requirements which are not directly concerned with the specific functions delivered by the system. They specify the criteria that can be used to judge the operation of a system rather than specific behaviors. They may relate to system properties such as reliability, performance, usability, scalability, and maintainability. Non-functional requirements arise through user needs, organizational policies, operational requirements, and performance expectations.

- **Performance:** The system should efficiently extract and process large volumes of Google Maps data with minimum response time and resource utilization.
- **Reliability:** The system should provide accurate and consistent extraction of business information, ensuring dependable operation during continuous scraping tasks.
- **Usability:** The application should provide a simple and user-friendly interface that allows users to easily enter search queries, extract data, and view results.
- **Scalability:** The system should support data collection from multiple locations and business categories without affecting performance.
- **Maintainability:** The project is implemented using Python and modular components, making it easy to update, debug, and enhance in the future.
- **Portability:** The application can run on Windows systems and can be adapted to other operating systems with minimal modifications.
- **Data Integrity:** The system should ensure that extracted data is properly validated, cleaned, and stored without loss of information.
- **Availability:** The system should remain stable and operational during long-duration scraping and data collection processes.

6.2.3 PERFORMANCE REQUIREMENT

- a) The system must provide fast and efficient data extraction from Google Maps, ensuring quick retrieval of business details such as names, addresses, phone numbers, ratings, and reviews with minimal processing time.
- b) The system should operate with minimal delay between user requests and system responses, enabling real-time search, scraping, data processing, and storage operations for improved user experience.
- c) The system should be reliable and capable of handling large volumes of data and multiple search queries simultaneously without data loss, extraction errors, or significant performance degradation.

6.3 INTRODUCTION TO PYTHON

Python is a powerful high-level, object-oriented programming language created by Guido van Rossum. Python is a general-purpose language. It has a wide range of applications from Web development (like: Django and Bottle), scientific and mathematical computing (Orange, SymPy, NumPy) to desktop graphical user Interfaces (Pygame, Panda3D).

6.3.1 INTRODUCTION TO SELENIUM

Selenium is an open-source automation tool used for controlling web browsers through programming languages such as Python, Java, and C#. It enables developers to automate browser actions such as opening web pages, clicking buttons, entering text, scrolling pages, and extracting information from websites. Selenium is widely used for web application testing and web scraping because it can interact with dynamic web content just like a human user.

In this project, Selenium is used to:

- Open and navigate Google Maps automatically.
- Enter search keywords and perform searches.
- Handle dynamic content loading and scrolling.
- Access business listings and details.
- Extract information such as names, addresses, phone numbers, ratings, and reviews.

6.3.2 INTRODUCTION TO BEAUTIFULSOUP

BeautifulSoup is a Python library used for parsing and extracting data from HTML and XML documents. It helps developers navigate webpage structures and retrieve specific information from web pages easily. BeautifulSoup converts complex HTML code into a readable format, making it simple to locate and extract required data elements.

In this project, BeautifulSoup is used to:

- Parse the HTML content of Google Maps pages.
- Extract business names and categories.

- Retrieve addresses and contact information.
- Collect ratings and review counts.
- Organize extracted data into a structured format.

6.4 SYSTEM DESIGN

Module Description

Entire project is divided into 4 modules as follows:

- User Input Module
- Google Maps Automation Module
- Data Extraction Module
- Data Processing and Storage Module

Module 1: User Input Module

- a. Users can enter business categories, keywords, or location names into the system.
- b. The system accepts user search queries and validates the input.
- c. Users can specify the type of business information they want to collect.
- d. The module initiates the data scraping process based on the provided search criteria.

Module 2: Google Maps Automation Module

- a. The system automatically opens and accesses Google Maps using Selenium.
- b. Search operations are performed based on user input.
- c. The module handles webpage navigation, scrolling, and dynamic content loading.
- d. Business listings are identified and prepared for data extraction.

Module 3: Data Extraction Module

- a. The system extracts business names, categories, and contact information.
- b. Addresses, ratings, review counts, and geographical coordinates are collected.
- c. BeautifulSoup or xml is used to parse webpage content and retrieve required information.
- d. Extracted data is validated to ensure accuracy and completeness.

Module 4: Data Processing and Storage Module

- a. The collected data is cleaned and organized into a structured format.
- b. Duplicate and unnecessary records are removed from the dataset.
- c. The processed information is stored in CSV files or databases.
- d. The stored data can be used for market research, lead generation.

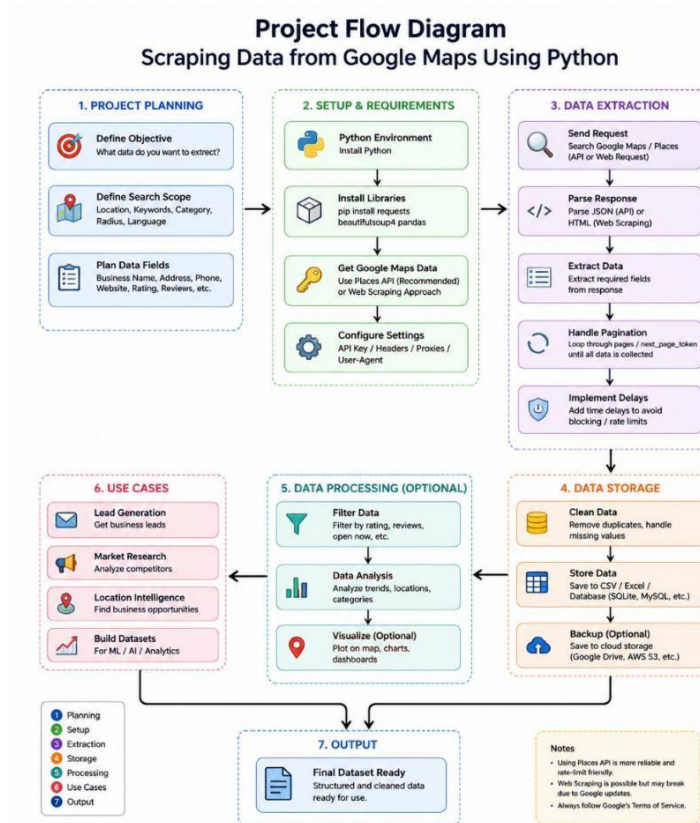


FIG 6.1. PROJECT FLOW

6.4.1 SYSTEM ARCHITECTURE

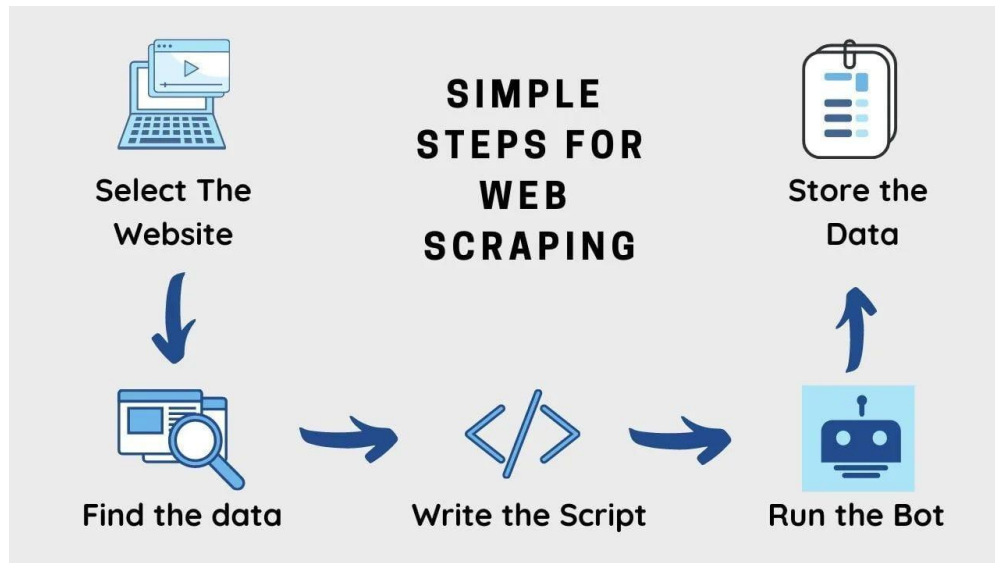


FIG 6.2. SYSTEM ARCHITECTURE

System Architecture Explanation

User Search Input: The system begins with the user entering business categories, keywords, or location names that need to be searched on Google Maps. These search queries act as the input for the data extraction process.

Google Maps Access: The system automatically accesses Google Maps using Selenium. It performs the required search operations and navigates through business listings based on the user-provided keywords and locations.

Browser Automation Process: Selenium automates browser activities such as entering search queries, scrolling through search results, handling dynamic content loading, and opening business profiles. This ensures smooth interaction with the Google Maps interface.

Data Extraction Process: After the required pages are loaded, BeautifulSoup or lxml parses the HTML content and extracts business details such as business names, addresses, phone numbers, ratings, review counts, categories, and geographical coordinates.

Data Processing and Validation: The extracted information is cleaned, filtered, and validated to remove duplicate or incomplete records. This process ensures the accuracy and consistency of the collected data.

Data Storage Module: The processed data is stored in structured formats such as CSV files or databases. Organized storage makes the information easy to access, analyze, and utilize for various business applications.

Data Access and Analysis: Users can access the stored datasets for market research, lead generation, competitor analysis, business intelligence, and location-based decision-making purposes.

Result Presentation: Finally, the system displays the extracted and processed business information to the user. The collected data can also be exported for further analysis, reporting, and decision-making activities.

6.5 DATA FLOW DIAGRAMS

DFDs are used to depict graphically the data flow in a system. They explain the processes involved from input to final output generation and show all possible paths between system entities. The level of detail in a data flow diagram is represented through three stages: Level 0 (context diagram), Level 1 (detailed view of major processes), and Level 2 (further decomposition of processes).

A DFD typically consists of four main components: processes, data flows, data stores, and external entities. Processes represent operations performed on data, data flows indicate the

movement of information, data stores represent where data is stored, and external entities denote sources or destinations outside the system.

DFDs help in understanding system functionality, improving communication between stakeholders, and identifying inefficiencies or redundancies in the system design. They also serve as a foundation for system analysis and design by providing a clear visualization of how data moves and transforms within the system.

6.5.1 DATA FLOW DIAGRAM LEVEL 0

DFD Level 0 gives a basic overview of the Google Maps Data Scraping System and its interaction with the user.

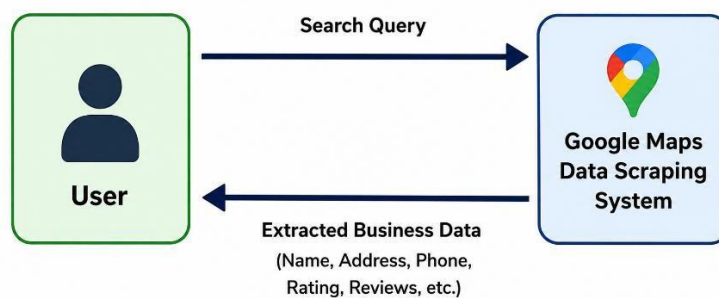


FIG 6.3. DFD LEVEL 0

6.5.2 DATA FLOW DIAGRAM LEVEL 1

DFD Level 1 provides a detailed view of the Google Maps Data Scraping System by breaking the entire process into major functional modules. The process begins when the user enters search keywords or location details through the User Input Module. The Google Maps Automation Module, implemented using Selenium, automatically performs searches and navigates through business listings. The Data Extraction Module uses BeautifulSoup to collect business information such as names, addresses, phone numbers, ratings, and reviews. The extracted data is then cleaned and validated in the Data Processing Module. Finally, the processed information is stored in CSV files or databases and displayed to the user as results.

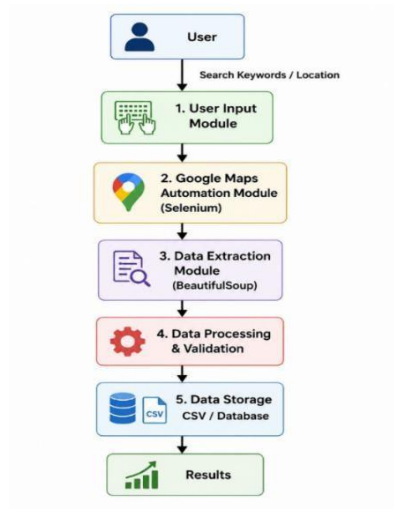


FIG 6.4. DFD LEVEL 1

6.5.3 DATA FLOW DIAGRAM LEVEL 2

DFD Level 2 provides a detailed representation of the internal working process of the Google Maps Data Scraping System. The process begins when the user enters business keywords or location details. The system performs a search on Google Maps and uses Selenium for browser automation, including opening Google Maps, entering search queries, scrolling through results, and accessing business listings. After the required pages are loaded, BeautifulSoup extracts business information such as business names, addresses, phone numbers, ratings, reviews, and geographical coordinates.



FIG 6.4. DFD LEVEL 2

6.6 UML DIAGRAMS

6.6.1 CLASS DIAGRAM

The Class Diagram illustrates the main classes and their relationships in the Google Maps Data Scrapping System. The User class represents the user who enters search queries, views extracted results, downloads reports, and logs out of the system. The Google Maps Scraper class performs the core functions of searching Google Maps, extracting business information, processing the collected data, and storing it. The Database class manages the storage of business data and generated reports. It provides methods for saving and retrieving information.

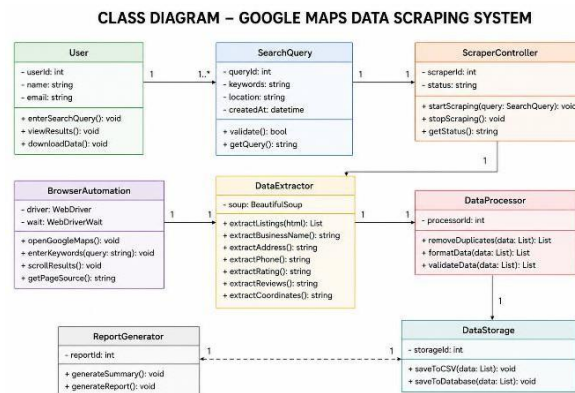


FIG 6.6. CLASS DIAGRAM

6.6.2 USE CASE DIAGRAM

The Use Case Diagram represents the interactions between the User and the Google Maps Data Scrapping System. It illustrates the main functionalities available to the user throughout the data extraction process. The user can log in to the system, enter search queries, search Google Maps, and extract business information such as names, addresses, phone numbers, ratings, and reviews.

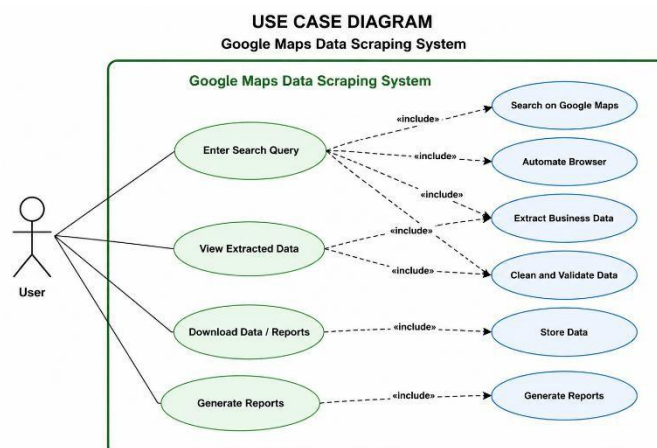


FIG 6.7. USE CASE DIAGRAM

6.6.3 SEQUENCE DIAGRAM

The Sequence Diagram illustrates the sequence of interactions between the **User** and the Google Maps Data Scraping System. The process begins when the user logs into the system and enters a search query. The system then performs a search on Google Maps and extracts the required business information.

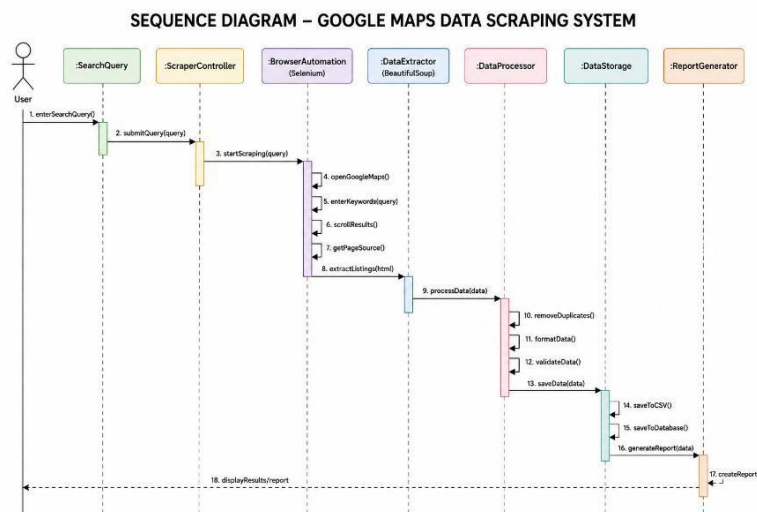


FIG 6.8. SEQUENCE DIAGRAM

6.6.4 COLLABORATIVE DIAGRAM

The Collaboration Diagram illustrates how the User and the Google Maps Data Scraping System interact to accomplish different tasks. The numbered messages represent the sequence of communication between the user and the system.

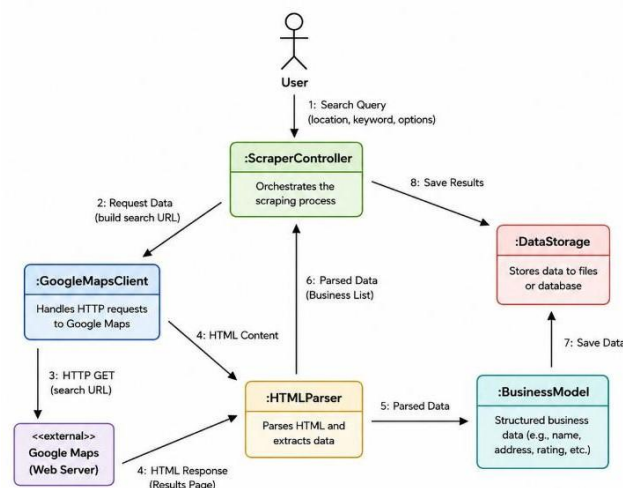


FIG 6.9. COLLABORATIVE DIAGRAM

CHAPTER-07

IMPLEMENTATION AND RESULT

7.1 INTRODUCTION

Implementation is the process of converting the proposed system design into a working application. It involves developing, integrating, and testing various modules to ensure that the system performs its intended functions effectively. In this project, the implementation phase focuses on developing an automated Google Maps Data Scraping System using Python and related web scraping technologies. The system is designed to collect business and location-based information efficiently and store it in a structured format for further analysis. Selenium for browser automation, and BeautifulSoup or xml for extracting information from web pages. The system automatically interacts with Google Maps, performs searches based on user input, navigates through search results, and extracts business details such as names, addresses, phone numbers, ratings, review counts, and geographical coordinates.

7.2 CODE FOR IMPLEMENTATION

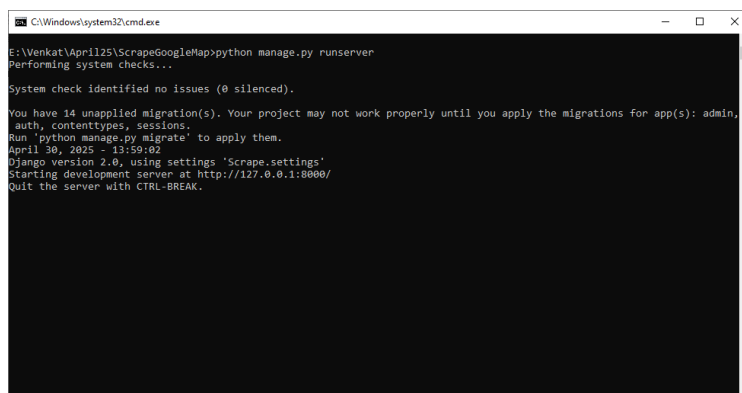
Code For Importing Libraries

```
pip install numpy==1.21.6
pip install pandas==1.3.5
pip install Django==2.1.7
pip install PyMySQL==0.9.3
pip install google-search-results==2.4.2
```

Code For Displaying Results

```
context = {'data': output}
return render(request, 'UserScreen.html', context)
```

7.3 OUTPUT



```
C:\Windows\system32\cmd.exe
E:\Venkat\April25\ScrapeGoogleMap>python manage.py runserver
Performing system checks...

System check identified no issues (0 silenced).

You have 14 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
April 30, 2025 - 13:59:02
Django version 2.0, using settings 'Scrape.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

FIG 7.1. PYTHON SERVER START

In above screen python server started and now open browser and enter URL as <http://127.0.0.1:8000/index.html> and then press enter key to get below page

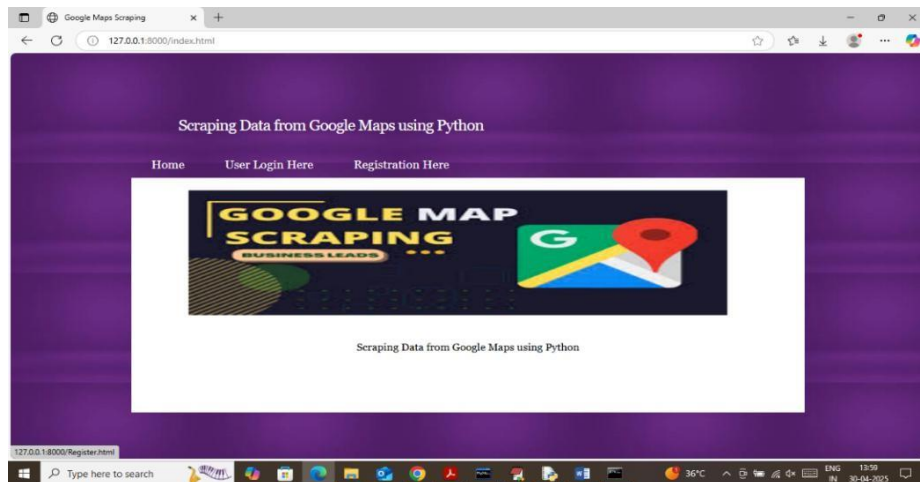


FIG 7.2. HOME PAGE

In above screen click on 'Registration Here' link to get below page

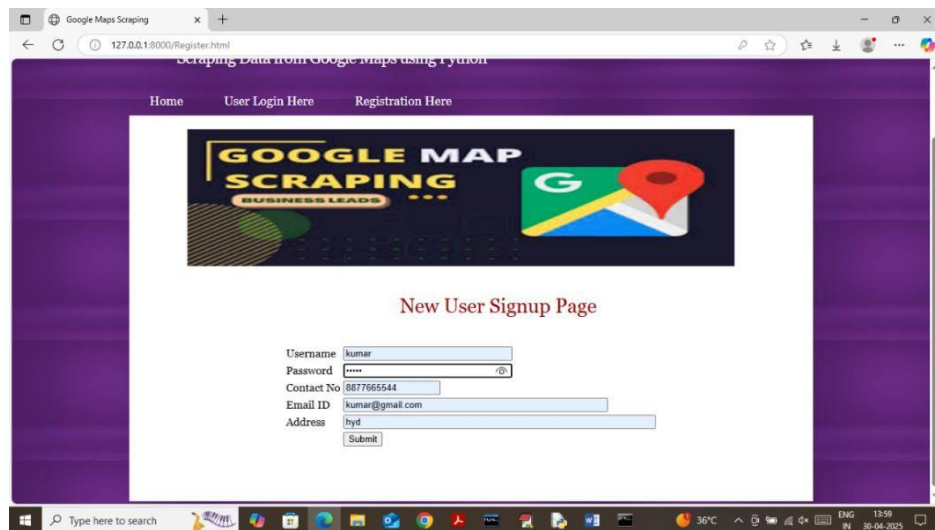


FIG 7.3. USER REGISTRATION

In above screen user is entering sign up details and then press button to get below page

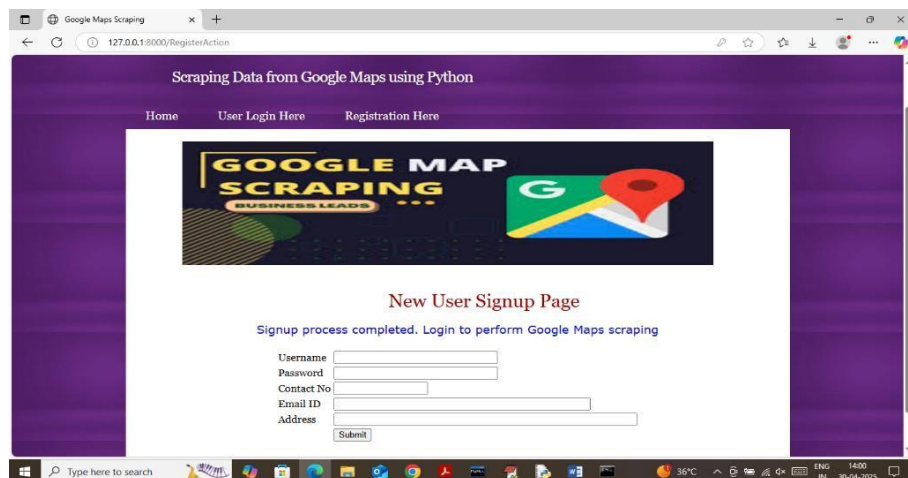


FIG 7.4. REGISTRATION SUCCESS

In above screen sign up process completed and now click on ‘User Login’ link to get below page.

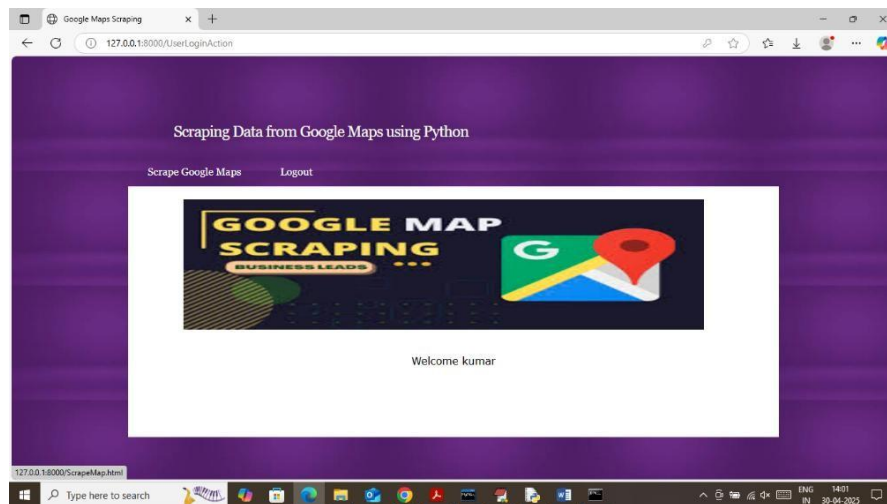


FIG 7.5. USER LOGIN

In above screen user is login and after login will get OTP in email like below page.

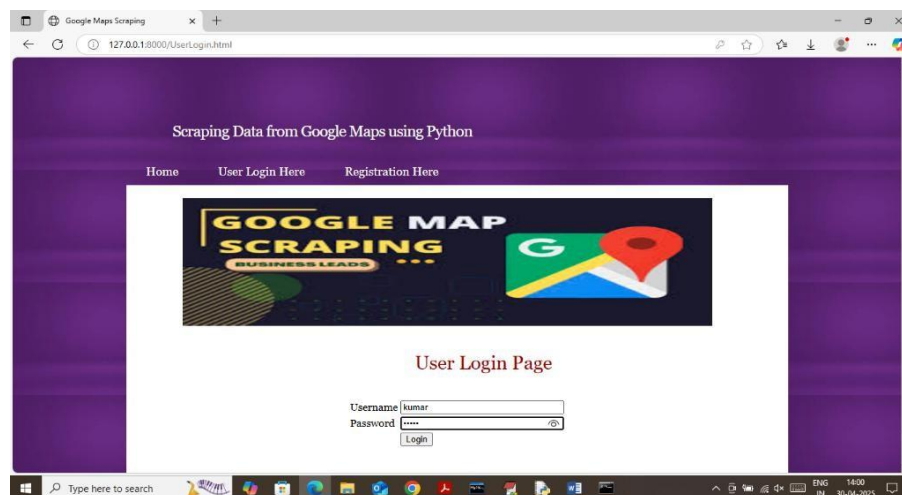


FIG 7.6. OTP VERIFICATION MAIL

In above screen OTP received to email and enter to application to continue authentication process.

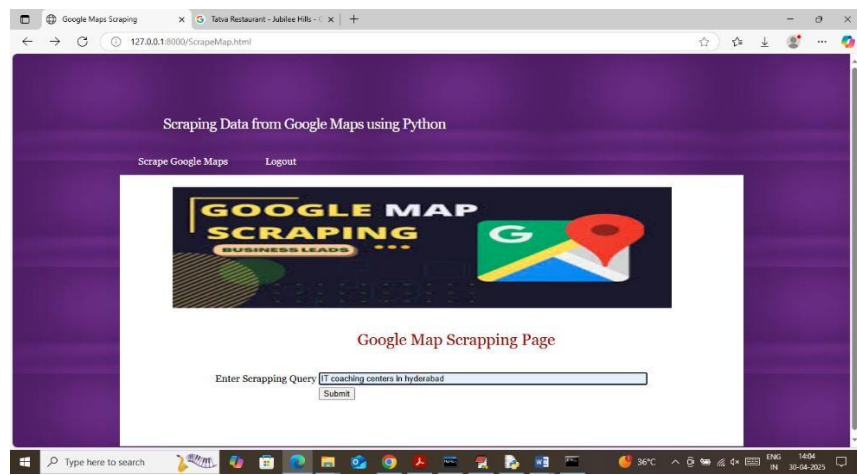


FIG 7.7. OTP AUTHENTICATION

In the above screen, the query 'IT Coaching Centers in Hyderabad' is entered, and the output is shown below

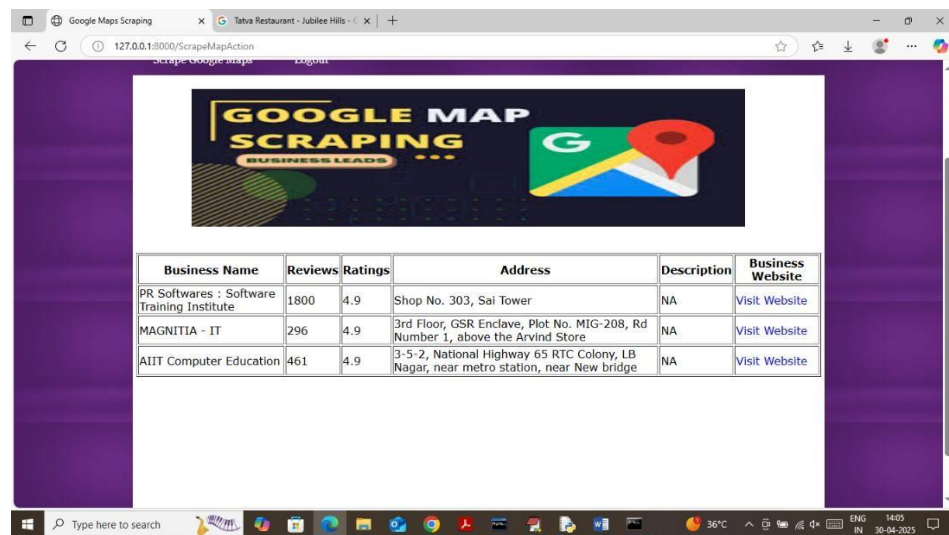


FIG 7.8. USER DASHBOARD

Similarly “Give Query and Get Output”

Note: as we know now-a-days Google services are not free of cost even for scrapping so when we search with free codes then Google will give few search results and in paid versions we can get many search outputs.

CHAPTER-8

CONCLUSIONS AND FUTURE ENHANCEMENT

8.1 CONCLUSION

The Google Maps Data Scraping System using Python provides an efficient and automated solution for extracting location-based and business-related information from Google Maps. The system utilizes Python libraries such as Selenium and BeautifulSoup to automate the process of data collection, reducing the need for manual effort and minimizing the chances of human error. It efficiently extracts valuable information such as business names, addresses, phone numbers, ratings, review counts, and geographical coordinates, which can be stored in structured formats for further analysis. It is capable of handling large volumes of data from multiple locations and business categories while ensuring accuracy and efficiency. the project demonstrates the effectiveness of web scraping technologies in automating data extraction tasks and transforming unstructured web information into useful datasets. The system improves productivity, saves time, and supports organizations in making informed decisions based on reliable location-based data.

8.2 FUTURE ENHANCEMENT

The future enhancement of the Google Maps Data Scraping System aims to improve its capabilities, efficiency, and usability by incorporating advanced technologies and additional features. As the demand for location-based information continues to grow, the system can be upgraded to support more intelligent and large-scale data collection processes. One of the major enhancements includes implementing automated scheduling features that allow users to perform data extraction tasks at specific intervals without manual intervention. This will help businesses and researchers maintain up-to-date datasets with minimal effort. The system can also be integrated with cloud storage and database platforms to manage and process large volumes of extracted data more efficiently. These technologies can be used to analyze collected business data, identify market trends, classify businesses, and generate valuable insights for decision-making. Advanced filtering and search mechanisms can also be incorporated to improve data accuracy and provide users with more precise results based on specific requirements

REFERENCES

- [1] Mitchell, R. (2018). *Web Scraping with Python: Collecting More Data from the Modern Web* (2nd Edition). O'Reilly Media.
- [2] Richardson, L. (2023). *Beautiful Soup Documentation: Screen-Scraping Library for Python*. Python Software Foundation.
- [3] SeleniumHQ. (2024). *Selenium WebDriver Documentation*. Selenium Project.
- [4] Van Rossum, G., & Drake, F. L. (2024). *Python Language Reference Manual*. Python Software Foundation.
- [5] Google Developers. (2024). *Google Maps Platform Documentation*. Google LLC.